



Signal and Image Processing MILSTONE 2

By:

(auto team)

mohamed Abdelhamid 13002323 T02

Haroun Ghoneim 13005136 T02

Andrew Elkess 13002881 T02

Rana Nagy 13007179 T08

Contents

Signal and Image Processing	1
1. Feature Extraction Overview	3
2. Tuning and Optimization	4
3. Results	5

GitHub link:

<https://github.com/medo-abdelhamid/Image-milestone-2-project.git>

1. Feature Extraction Overview

To make the model as accurate as possible, I combined several different types of Feature Extraction to capture every detail of the balls. Here is the full list of features used:

- **Color Histograms:** I used both 3D RGB histograms and 1D grayscale histograms to capture the primary colors of the balls.
- **GLCM:** Gray-Level Co-occurrence Matrix features (contrast, dissimilarity, homogeneity, energy, and correlation) were used to identify surface textures.
- **HOG:** Histograms of Oriented Gradients to capture the overall structure and directional edges.
- **SIFT:** Scale Invariant Feature Transform to find unique "keypoints" on the balls that don't change even if the ball is rotated.
- **Contour Features:** I calculated the number of contours, the area, and the "circularity" (how close the shape is to a perfect circle).
- **DoG:** Difference of Gaussians was used to highlight fine edges and details that might be missed in a normal grayscale image.
- **K-Means Features:** I used K-means clustering on the pixel values to find the dominant colors and their proportions.
- **Hough Circle Transform:** This was a specific addition to detect the radius and center of the balls.

The addition of the **Hough Transform** came from my mechatronics engineering project. I am working on a **lane detection project** where I used the Hough Line Transform to find lane markings on the road. While researching that, I found out there is also a **Hough Circle Transform**. Since all the objects in this dataset (balls) are circular, I realized this would be the perfect way to give the model hint about the shape and size of the object it's looking at.

2. Tuning and Optimization

The final high accuracy didn't happen on the first try. I had to experiment with several parameters to get the best results:

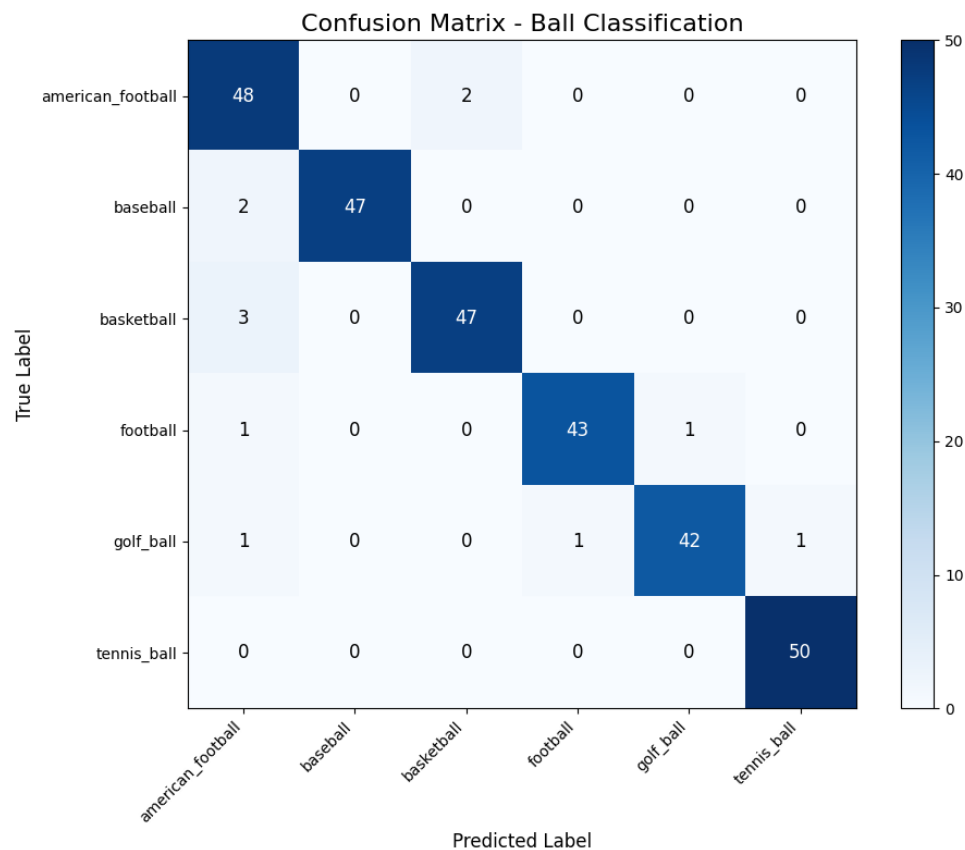
- **Image Resizing:** I originally started with a resize of **128\ 128**, but the accuracy was not great because the images lost too much detail. Once I increased the resolution to **256 \ 256**, the accuracy improved significantly as the texture features became much clearer.
- **SIFT "N-Features" Tuning:** I spent a lot of time playing with the number of SIFT. I initially tried 100 features, but the model was performing worse. After testing different values, I found that 65 features were the good that gave me the maximum accuracy.
- **Model Comparison:** I didn't just use Random Forest from the start. I tried **SVM** and **k-NN** first. However, the accuracy with those models was quite low. The feature vector I created was very large and complex, and Random Forest (with 500 trees) was much better at handling that complexity than the other two.

3. Results

Vs code was used to run this code .. final accuracy=95.85%

```
lahza wahda ma3alesh..
dataset-balls/train/american_football
dataset-balls/train/baseball
dataset-balls/train/basketball
dataset-balls/train/football
dataset-balls/train/golf_ball
dataset-balls/train/tennis_ball
tamm.
estany bat3alem ahoo ya basha, shwaya shwaya
tamam ya basha!

ya mosahel
dataset-balls/test/american_football
dataset-balls/test/baseball
dataset-balls/test/basketball
dataset-balls/test/football
dataset-balls/test/golf_ball
dataset-balls/test/tennis_ball
```



Auto team

Final Accuracy with Random Forest: 95.85

	precision	recall	f1-score	support
american_football	0.87	0.96	0.91	50
baseball	1.00	0.96	0.98	49
basketball	0.96	0.94	0.95	50
football	0.98	0.96	0.97	45
golf_ball	0.98	0.93	0.95	45
tennis_ball	0.98	1.00	0.99	50
accuracy			0.96	289
macro avg	0.96	0.96	0.96	289
weighted avg	0.96	0.96	0.96	289

Confusion Matrix:

```
[[48  0  2  0  0  0]
 [ 2 47  0  0  0  0]
 [ 3  0 47  0  0  0]
 [ 1  0  0 43  1  0]
 [ 1  0  0  1 42  1]
 [ 0  0  0  0  0 50]]
```

□